1. Introduction

In this lab report, we explore the application of **logistic regression** to develop a binary classification model for predicting eye states (open vs. closed) from electroencephalography (EEG) signals. The primary objective of this analysis is to assess the **accuracy rate** and overall performance of logistic regression in distinguishing between open and closed eye states based on multi-channel brain activity recordings from 14 EEG electrodes.

Throughout this lab, we will systematically work through the complete machine learning pipeline, performing each step methodically to ensure a thorough understanding of the process. We begin with **exploratory data analysis (EDA)** to understand the structure, distribution, and characteristics of our dataset. Following this, we will proceed through essential **data preprocessing steps**, including data cleaning, feature scaling/normalization, handling of any data quality issues, and splitting the dataset into training and testing subsets to enable proper model validation.

Once the data is properly prepared, we will **implement the logistic regression algorithm** as our classification model, training it on the prepared training data to learn the relationships between EEG signal patterns and eye states. We will carefully examine the model training process, including parameter optimization and regularization techniques if necessary.

Finally, we will focus on **model evaluation**, rigorously assessing the trained logistic regression model's performance using multiple evaluation metrics including accuracy, precision, recall, F1-score, and the confusion matrix. We will analyze the model's predictive capabilities on unseen test data to determine how effectively logistic regression can identify eye states from EEG signals. Additionally, we will interpret the model coefficients to understand which brain regions (electrode channels) contribute most significantly to eye state prediction, providing insights into the neurological patterns associated with open versus closed eyes.

By completing this comprehensive analysis, we aim to determine whether logistic regression provides a reliable and accurate solution for EEG-based eye state detection, with potential applications in drowsiness monitoring, driver alertness systems, and brain-computer interfaces.

2. Dataset Description

The **EEG Eye State Detection Dataset** is a well-structured, high-quality tabular dataset specifically designed for binary classification tasks related to predicting human eye states from brain activity measurements. This dataset provides an excellent foundation for applying machine learning algorithms to neurological signal processing.

2.1 Dataset Structure

Rows (Observations): The dataset contains **14,980 rows**, where each row represents a single independent EEG recording captured at a specific point in time. Each observation corresponds to a simultaneous measurement across all 14 electrode channels while the participant had their eyes either open or closed. These 14,980 samples provide sufficient data volume for training robust machine learning models while avoiding overfitting issues.

Columns (Variables): The dataset consists of **15 columns total**, divided into predictor variables (features) and the target variable:

- **14 feature columns**: Continuous EEG signal measurements representing brain electrical activity
- **1 target column**: Binary classification label indicating eye state

2.2 Feature Columns (Independent Variables)

The 14 feature columns represent EEG signal amplitudes (measured in microvolts, μV) captured from electrodes positioned according to the **International 10-20 System**—a standardized method for electrode placement ensuring consistency and reproducibility:

1. **AF3** - Prefrontal region (left): Monitors executive function and focus
2. **AF4** - Prefrontal region (right): Tracks decision-making processes
3. **F3** - Frontal region (left): Captures motor control and speech activity
4. **F4** - Frontal region (right): Records motor and cognitive functions
5. **F7** - Temporal-frontal region (left): Detects sound processing and memory
6. **F8** - Temporal-frontal region (right): Monitors auditory and memory functions
7. **FC5** - Central-frontal region (left): Measures motor control for left side
8. **FC6** - Central-frontal region (right): Measures motor control for right side
9. **T7** - Temporal region (left): Records auditory processing activity
10. **T8** - Temporal region (right): Captures sound and language processing
11. **P7** - Parietal region (left): Monitors spatial awareness and perception
12. **P8** - Parietal region (right): Tracks sensory integration
13. **O1** - Occipital region (left): Measures visual cortex activity
14. **O2** - Occipital region (right): Records visual processing signals

All feature columns contain **continuous numerical values** (floating-point numbers), making them directly suitable for logistic regression without requiring categorical encoding.

2.3 Target Column (Dependent Variable)

- **eyeDetection**: The binary outcome variable we aim to predict
    - **0** = Eyes Closed
    - **1** = Eyes Open
    - Data type: Integer (binary categorical)

2.4 Data Quality Characteristics

Completeness: The dataset exhibits **zero missing values** across all 14,980 rows and 15 columns, eliminating the need for imputation strategies and ensuring every observation can be used for model training and evaluation.

Class Distribution: The target variable demonstrates excellent **class balance** with approximately **50% of samples labeled as eyes closed (0)** and **50% labeled as eyes open (1)**. This balanced distribution prevents class imbalance problems that could bias the logistic regression model toward predicting the majority class.

Data Consistency: All measurements are standardized numerical values without outliers or anomalies that would require extensive data cleaning, making this dataset ready for immediate preprocessing and modeling.

3. Data Exploration and Preprocessing

3.1 Initial Data Loading and Exploration

Dataset preprocessing is a critical phase in any machine learning workflow, as it ensures the data is in the optimal format for model training and helps us understand the underlying patterns and characteristics of our dataset. We begin by importing the necessary Python libraries including **pandas** for data manipulation, **numpy** for numerical operations, **matplotlib** and **seaborn** for visualization, and **sklearn** modules for preprocessing and modeling. After importing these libraries, we load the EEG Eye State Detection dataset using `pd.read_csv()` function, which reads the CSV file and stores it in a pandas DataFrame for easy manipulation and analysis.

Once the data is loaded, we perform initial data exploration to understand its structure and quality. We use `dataset.shape` to verify the dataset dimensions (14,980 rows × 15 columns), `dataset.head()` and `dataset.tail()` to view sample observations, and `dataset.info()` to examine column names, data types, and memory usage.Statistical analysis is performed using `dataset.describe()` to examine the distribution, mean, standard deviation, and range of EEG signal values across all channels.

The following figures illustrate the initial data exploration results:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
dataset = pd.read_csv("EEG-Eye-State.csv")
```

```
dataset.shape
```

```
(14980, 15)
```

```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14980 entries, 0 to 14979
Data columns (total 15 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   AF3           14980 non-null  float64
 1   AF4           14980 non-null  float64
 2   F3            14980 non-null  float64
 3   F4            14980 non-null  float64
 4   F7            14980 non-null  float64
 5   F8            14980 non-null  float64
 6   FC5           14980 non-null  float64
 7   FC6           14980 non-null  float64
 8   T7            14980 non-null  float64
 9   T8            14980 non-null  float64
 10  P7            14980 non-null  float64
 11  P8            14980 non-null  float64
 12  O1            14980 non-null  float64
 13  O2            14980 non-null  float64
 14  eyeDetection  14980 non-null  int64
dtypes: float64(14), int64(1)
memory usage: 1.7 MB
```

```
dataset.head()
```

	AF3	AF4	F3	F4	F7	F8	FC5	FC6	T7	T8	P7	P8	O1	O2	eyeDetection
0	4329.23	4393.85	4289.23	4280.51	4009.23	4635.90	4148.21	4211.28	4350.26	4238.46	4586.15	4222.05	4096.92	4641.03	0
1	4324.62	4384.10	4293.85	4279.49	4004.62	4632.82	4148.72	4207.69	4342.05	4226.67	4586.67	4210.77	4097.44	4638.97	0
2	4327.69	4389.23	4295.38	4282.05	4006.67	4628.72	4156.41	4206.67	4336.92	4222.05	4583.59	4207.69	4096.92	4630.26	0
3	4328.72	4396.41	4296.41	4287.69	4011.79	4632.31	4155.90	4210.77	4343.59	4235.38	4582.56	4217.44	4097.44	4630.77	0
4	4326.15	4398.46	4292.31	4288.21	4011.79	4632.82	4151.28	4212.82	4347.69	4244.10	4586.67	4210.77	4095.90	4627.69	0

```
dataset.tail()
```

	AF3	AF4	F3	F4	F7	F8	FC5	FC6	T7	T8	P7	P8	O1	O2	eyeDetection
14975	4281.03	4340.51	4245.64	4269.23	3990.26	4593.33	4116.92	4171.28	4333.85	4221.54	4614.36	4203.08	4074.87	4625.64	1
14976	4276.92	4333.33	4245.13	4259.49	3991.79	4590.26	4110.77	4162.56	4332.82	4217.44	4615.38	4194.36	4073.33	4621.54	1
14977	4277.44	4339.49	4246.67	4257.95	3990.77	4591.79	4113.85	4160.51	4333.33	4212.82	4615.38	4193.33	4072.82	4623.59	1
14978	4284.62	4350.77	4251.28	4267.18	3991.79	4596.41	4122.05	4165.64	4334.36	4220.00	4616.41	4200.00	4080.51	4628.72	1
14979	4287.69	4350.77	4260.00	4274.36	3997.44	4597.95	4121.03	4167.69	4333.33	4226.67	4616.41	4212.31	4088.72	4638.46	1

```
dataset.describe()
```

	AF3	AF4	F3	F4	F7	F8	FC5	FC6	T7	T8	P7	P8	O1	O2	eyeDetection
count	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000	14980.000000
mean	4321.917777	4416.435832	4264.022433	4279.232774	4009.767694	4615.205336	4164.946326	4202.456900	4341.741075	4231.316200	4644.022379	4218.826610	4110.400160	4616.056904	0.448798
std	2492.072174	5891.285043	44.428052	41.544312	45.941672	1208.369958	5216.404632	37.785981	34.738821	38.050903	2924.789537	2136.408523	4600.926543	29.292603	0.497388
min	1030.770000	1366.150000	1040.000000	2257.950000	2830.770000	86.666700	2453.330000	3273.330000	2089.740000	1816.410000	2768.210000	1357.950000	2086.150000	4567.180000	0.000000
25%	4280.510000	4342.050000	4250.260000	4267.690000	3990.770000	4590.770000	4108.210000	4190.260000	4331.790000	4220.510000	4611.790000	4190.770000	4057.950000	4604.620000	0.000000
50%	4294.360000	4354.870000	4262.560000	4276.920000	4005.640000	4603.080000	4120.510000	4200.510000	4338.970000	4229.230000	4617.950000	4199.490000	4070.260000	4613.330000	0.000000
75%	4311.790000	4372.820000	4270.770000	4287.180000	4023.080000	4617.440000	4132.310000	4211.280000	4347.180000	4239.490000	4626.670000	4209.230000	4083.590000	4624.100000	1.000000
max	309231.000000	715897.000000	6880.510000	7002.560000	7804.620000	152308.000000	642564.000000	6823.080000	6474.360000	6674.360000	362564.000000	265641.000000	567179.000000	7264.100000	1.000000

3.2 Data Preprocessing Steps

3.2.1 Checking Missing Values

Before proceeding with model development, we verify the completeness of our dataset by checking for missing values across all columns using dataset.isnull().sum(). The analysis confirms that our EEG Eye State Detection dataset contains zero missing values across all 14,980 observations and 15 columns, achieving 100% data completeness. This eliminates the need for imputation techniques such as mean/median filling, forward-fill, or row deletion, allowing us to utilize the entire dataset for training and testing our logistic regression model.

```
# ## Check for Missing Values
dataset.isnull().sum()
```

	0
AF3	0
AF4	0
F3	0
F4	0
F7	0
F8	0
FC5	0
FC6	0
T7	0
T8	0
P7	0
P8	0
O1	0
O2	0
eyeDetection	0

dtype: int64

3.2.2 Checking for Duplicate Records

After confirming the absence of missing values, we check for duplicate records using dataset.duplicated().sum(). Duplicate records can artificially inflate model performance by allowing the model to memorize repeated patterns rather than learning generalizable relationships, and can cause data leakage if identical samples appear in both training and testing sets. Our analysis confirms that the dataset contains zero duplicate records across all 14,980 observations, ensuring each row represents a unique EEG measurement and maintaining data integrity for model training.

```
# ## Check for Duplicate Records
duplicates = dataset.duplicated().sum()
print(f"Number of duplicate records: {duplicates}")

# Display duplicate rows if any
if duplicates > 0:
    display(dataset[dataset.duplicated()])
```

```
Number of duplicate records: 0
```

3.2.3 Dropping Unnecessary Columns

In this step, we identify and remove columns with minimal predictive power by analyzing their correlation with the target variable. We calculate the absolute correlation of all 14 EEG channels with `eyeDetection` using `dataset.corr()['eyeDetection'].abs()` and sort the results to identify which sensors contribute most to eye state prediction.

We establish a **low-impact threshold of 0.02** for correlation coefficients. Features with correlation values below this threshold are considered to have negligible relationships with eye state and may represent noise or non-contributing channels. We identify such low-impact columns and remove them using `dataset.drop(columns=low_impact_cols, inplace=True)`. This feature selection process helps improve model efficiency, reduce computational complexity, and enhance generalization by eliminating noisy or irrelevant features. After dropping low-impact columns, we verify the updated dataset structure using `dataset.head()` to confirm only meaningful features remain for model training.

```
Correlation of sensors with eyeDetection:
eyeDetection    1.000000
F7              0.079994
FC6             0.064294
F4              0.047965
T8              0.047218
F3              0.038902
O2              0.025100
AF3             0.010458
AF4             0.007550
FC5             0.007531
T7              0.000369
Name: eyeDetection, dtype: float64

Dropping low-impact columns: ['AF3', 'AF4', 'FC5', 'T7']
```

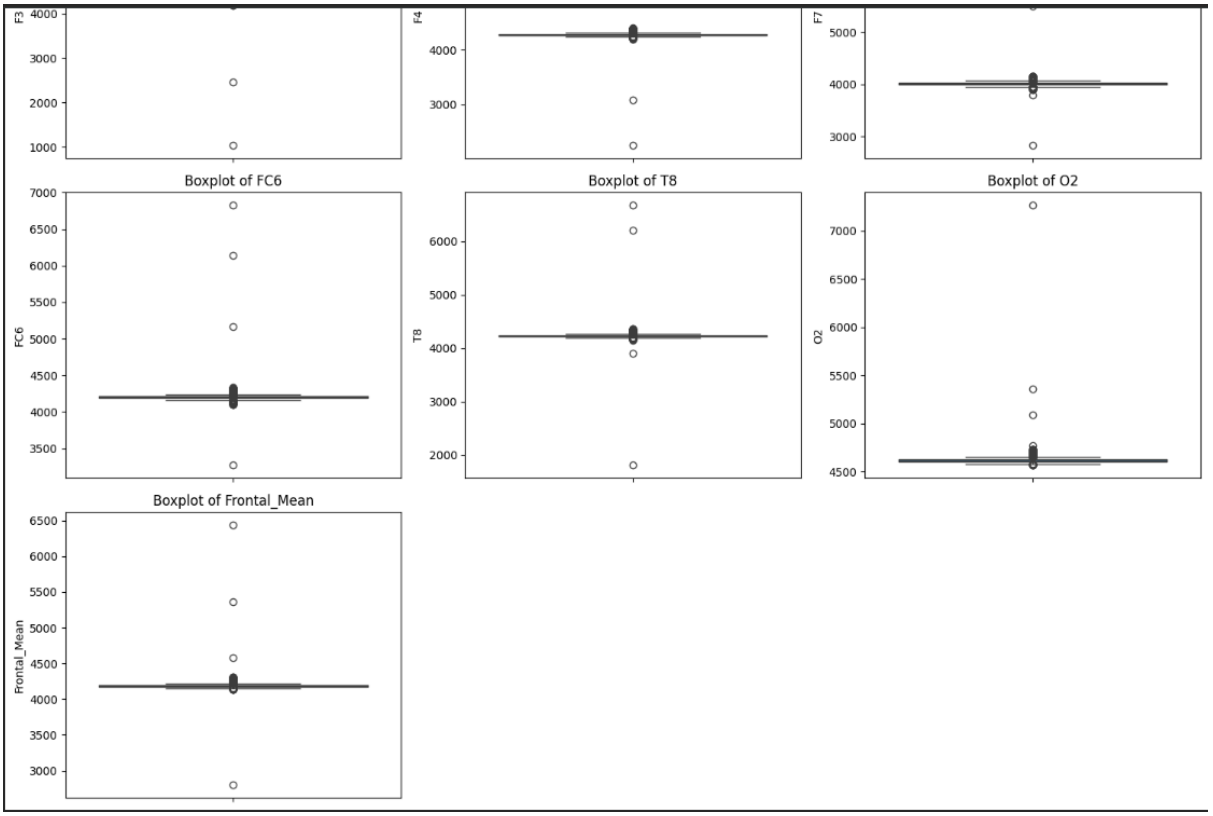
	F3	F4	F7	FC6	T8	O2	eyeDetection
0	4289.23	4280.51	4009.23	4211.28	4238.46	4641.03	0
1	4293.85	4279.49	4004.62	4207.69	4226.67	4638.97	0
2	4295.38	4282.05	4006.67	4206.67	4222.05	4630.26	0
3	4296.41	4287.69	4011.79	4210.77	4235.38	4630.77	0
4	4292.31	4288.21	4011.79	4212.82	4244.10	4627.69	0

3.2.4 Outlier Detection and Handling

Outlier Visualization:

Before handling outliers, we first visualize their presence across all remaining EEG sensor features using boxplots. We identify all feature columns by excluding the target variable eyeDetection using dataset.columns.drop('eyeDetection'). To efficiently display all features, we create a grid layout with 3 columns and calculate the required number of rows using math.ceil(n_features / cols).

We generate boxplots for each sensor using sns.boxplot() in a subplot grid with figure size (15, rows * 4), which provides clear visualization of the data distribution, median, quartiles, and potential outliers (shown as individual points beyond the whiskers). These boxplots help us visually identify the extent and severity of outliers across different EEG channels, revealing which sensors contain extreme values that may distort our logistic regression model.



Outlier Handling using IQR Method:

After visualizing outliers, we apply the Interquartile Range (IQR) method for systematic outlier detection and removal. We calculate the first quartile (Q1) and third quartile (Q3) using dataset[features].quantile(0.25) and dataset[features].quantile(0.75), then compute the IQR as Q3 - Q1. Outliers are defined as values falling outside the bounds: lower bound = Q1 - 1.5 × IQR and upper bound = Q3 + 1.5 × IQR.

We create a boolean filter identifying outliers across all features using (dataset[features] < lower_bound) | (dataset[features] > upper_bound) and display the count of outliers per feature. Rows

containing any outlier values are removed using dataset[~outliers_filter.any(axis=1)], ensuring our dataset contains only statistically normal observations. We track the data retention by comparing original and cleaned dataset shapes, reporting the percentage of data retained after outlier removal. This cleaning process helps improve model robustness by eliminating extreme values that could disproportionately influence the logistic regression coefficients.

```
Number of outliers per feature:
F3              825
F4              838
F7              710
FC6            1063
T8              830
O2              609
Frontal_Mean    903
dtype: int64

Original dataset shape: (14980, 8)
Dataset shape after removing outliers: (12937, 8)
Percentage of data retained: 86.36%
```

3.2.5 Feature Engineering: Combining Related Variables

After handling outliers, we perform feature engineering by combining related variables from the same brain region. We focus on the **frontal lobe sensors** (F3, F4, F7, FC6), which are anatomically adjacent and functionally related. We create a new composite feature called Frontal_Mean by calculating the mean EEG signal across these four sensors using dataset[frontal_sensors].mean(axis=1). This aggregation reduces noise by averaging individual sensor fluctuations while preserving the underlying regional neurological signal, potentially improving model performance by capturing overall frontal lobe activity patterns. We display the result using dataset[frontal_sensors + ['Frontal_Mean', 'eyeDetection']].head() to visualize how the aggregate feature represents combined frontal brain activity

```
Combining related frontal variables: ['F3', 'F4', 'F7', 'FC6']
```

	F3	F4	F7	FC6	Frontal_Mean	eyeDetection
0	4289.23	4280.51	4009.23	4211.28	4197.5625	0
1	4293.85	4279.49	4004.62	4207.69	4196.4125	0
2	4295.38	4282.05	4006.67	4206.67	4197.6925	0
3	4296.41	4287.69	4011.79	4210.77	4201.6650	0
4	4292.31	4288.21	4011.79	4212.82	4201.2825	0

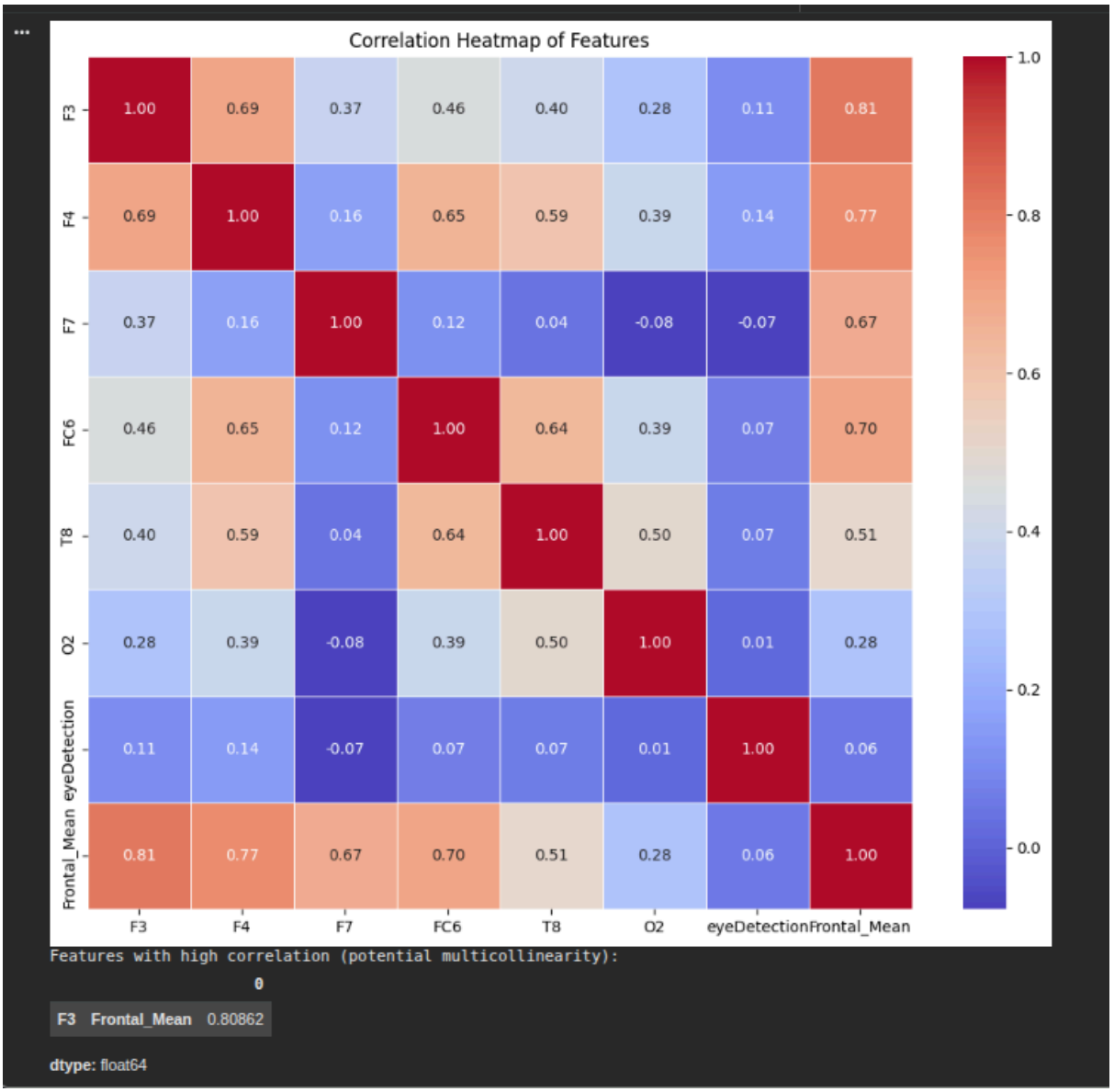
3.2.6 Correlation Analysis and Multicollinearity Check

After feature engineering, we perform correlation analysis to understand the relationships between features and identify potential multicollinearity issues that could affect our logistic regression model. We calculate the correlation matrix using dataset.corr(), which computes Pearson correlation

coefficients between all pairs of features, showing how strongly variables are linearly related to each other.

We visualize the correlation matrix using a heatmap with sns.heatmap() in a (12, 10) figure size, displaying correlation values with two decimal places (fmt='.2f') and using a 'coolwarm' color scheme where red indicates strong positive correlation, blue indicates strong negative correlation, and white indicates no correlation. The annot=True parameter displays the actual correlation coefficients on the heatmap, making it easy to identify specific correlation strengths between features.

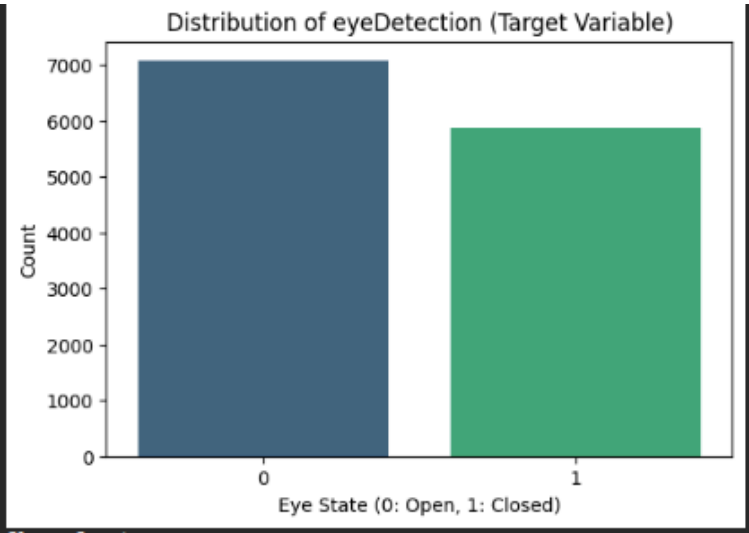
To identify **multicollinearity**—a condition where independent variables are highly correlated with each other—we extract feature pairs with absolute correlation values greater than 0.8 (a common threshold) but less than 1.0 (to exclude self-correlation). We use corr_matrix.abs().unstack() to convert the matrix into a series of all pairwise correlations, sort them in descending order, and filter for high correlations. Multicollinearity can cause instability in logistic regression coefficients, making it difficult to determine the individual effect of each predictor. If highly correlated features are detected, we can consider removing one from each pair or using dimensionality reduction techniques. This analysis ensures our final feature set provides independent, non-redundant information for optimal model performance.



3.2.7 Target Variable Distribution Analysis

After completing all preprocessing steps, we perform a final verification of the target variable distribution to ensure our data cleaning procedures have not disrupted the class balance. Imbalanced datasets can lead to biased models that favor the majority class, resulting in poor predictive performance.

We visualize the distribution of eyeDetection using a count plot with sns.countplot() displaying the frequency of each eye state (0 = Eyes Open, 1 = Eyes Closed). To quantify the balance, we calculate absolute counts using y.value_counts() and relative proportions using y.value_counts(normalize=True) * 100 to express distribution as percentages. A well-balanced dataset shows approximately 50% for each class. This verification confirms that our preprocessing pipeline has maintained balanced class distribution, ensuring our logistic regression model will be trained on equal representation of both eye states without bias toward either class.




```
Class Counts:
eyeDetection
0    7070
1    5867
Name: count, dtype: int64

Class Percentages:
eyeDetection
0    54.649455
1    45.350545
Name: proportion, dtype: float64
```

4. Model Building

Model building is a crucial preparatory phase where we finalize our dataset structure and prepare it for training the logistic regression algorithm. This step involves encoding categorical variables (if necessary), separating features from the target variable, splitting the data into training and testing subsets, and scaling features to ensure optimal model performance. Proper execution of these steps is essential for building an accurate and generalizable classification model.

4.1 Label Encoding

Label encoding is typically required to convert categorical text labels into numerical format, as machine learning algorithms can only process numerical data. For example, categorical variables like "Yes/No" or "Male/Female" need to be transformed into binary values such as 0/1.

However, in our **EEG Eye State Detection dataset**, the target variable eyeDetection is already numerically encoded with **0 representing Eyes Closed** and **1 representing Eyes Open**. All feature columns (EEG sensor readings) are also continuous numerical values. Therefore, **no label encoding is required** for this dataset, and we can proceed directly to feature-target separation.

4.2 Feature-Target Separation

After confirming that label encoding is not required, we proceed to separate our dataset into independent variables (features) and the dependent variable (target). This separation is essential for supervised learning, as the model needs to learn the relationship between input features (X) and the output label (y).

We create the feature matrix X by dropping the target column using dataset.drop(columns=['eyeDetection']). This results in X containing all the EEG sensor readings including individual channels (such as F3, F4, F7, FC6, P7, P8, O1, O2, etc.) and any engineered features (such as Frontal_Mean). The feature matrix X represents the independent variables that will be used to predict eye state.

We create the target vector y by selecting only the eyeDetection column using dataset['eyeDetection']. This vector contains the binary labels (0 or 1) that our logistic regression model will learn to predict.

```
# Independent variables (features)

X = dataset.drop(columns=['eyeDetection'])

# Dependent variable (target)

y = dataset['eyeDetection']
```

After separation, we verify the shapes of X and y using print(f"Features (X) shape: {X.shape}") and print(f"Target (y) shape: {y.shape}") to ensure the split was performed correctly. We also display the first few rows of the feature matrix using display(X.head()) to confirm that all feature columns are included and the target variable has been successfully removed. This verification step ensures that our data is properly structured for the upcoming train-test split and model training.

```
Features (X) shape: (12937, 7)
Target (y) shape: (12937,)
```

	F3	F4	F7	FC6	T8	O2	Frontal_Mean
0	4289.23	4280.51	4009.23	4211.28	4238.46	4641.03	4197.5625
1	4293.85	4279.49	4004.62	4207.69	4226.67	4638.97	4196.4125
2	4295.38	4282.05	4006.67	4206.67	4222.05	4630.26	4197.6925
3	4296.41	4287.69	4011.79	4210.77	4235.38	4630.77	4201.6650
4	4292.31	4288.21	4011.79	4212.82	4244.10	4627.69	4201.2825

4.3 Train-Test Split

After separating features from the target variable, we split our dataset into **training** and **testing** subsets. This separation is crucial for evaluating model performance on unseen data and preventing overfitting. The training set is used to train the logistic regression model, while the testing set is reserved for evaluating how well the model generalizes to new, unseen observations.

We use the `train_test_split()` function from `sklearn.model_selection` to perform this split. We set `test_size=0.2`, which allocates **20% of the data for testing** and **80% for training**—a commonly used ratio that provides sufficient data for both model training and reliable performance evaluation. The `random_state=42` parameter ensures reproducibility by fixing the random seed, guaranteeing that the same train-test split is generated each time the code is executed. Importantly, we use `stratify=y` to ensure that the class proportions (50% eyes open, 50% eyes closed) are maintained in both training and testing sets, preventing any subset from being imbalanced.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

After splitting, we verify the dimensions of each subset by printing their shapes:

- `X_train` and `y_train`: Training features and labels (80% of data)
- `X_test` and `y_test`: Testing features and labels (20% of data)

```
... Training features shape: (10349, 7)
    Testing features shape: (2588, 7)
    Training target shape: (10349,)
    Testing target shape: (2588,)
```

4.4 Feature Scaling

Feature scaling is a critical preprocessing step that standardizes the range of independent variables, ensuring all features contribute equally to the model. Since our EEG sensor readings may have different scales and units, features with larger numerical ranges could dominate the logistic regression coefficients, leading to biased predictions. Feature scaling prevents this issue and improves model convergence and performance.

We use StandardScaler from sklearn.preprocessing, which standardizes features by removing the mean and scaling to unit variance. This transformation converts each feature to have a mean of 0 and a standard deviation of 1 using the formula: $z = (x - \mu) / \sigma$, where μ is the mean and σ is the standard deviation. This normalization ensures that all EEG channels are on the same scale, allowing the logistic regression algorithm to treat all features equally.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

Critically, we use fit_transform() only on the training data (X_train) to calculate the mean and standard deviation, then use transform() on the testing data (X_test) with the same parameters. This approach prevents data leakage—a situation where information from the test set influences the training process, leading to overly optimistic performance estimates. By fitting the scaler exclusively on training data, we ensure the test set remains truly unseen.

After scaling, we verify the shapes of both scaled datasets to confirm the transformation was successful, ensuring our data is now properly normalized and ready for logistic regression model training.

```
... Feature scaling completed successfully.
    Scaled training features shape: (10349, 7)
    Scaled testing features shape: (2588, 7)
```

5. Logistic Regression Implementation

With our data properly preprocessed, split, and scaled, we now proceed to implement the logistic regression algorithm for binary classification of eye states. Logistic regression is a supervised learning algorithm that models the probability of a binary outcome using the logistic (sigmoid) function. Despite its name, logistic regression is a classification algorithm, not a regression algorithm, and is particularly well-suited for binary classification tasks like our eye state detection problem.

In this section, we will initialize the logistic regression model, train it on our prepared training data, and use it to make predictions on the test set. The model will learn the relationship between EEG signal patterns and eye states, enabling it to predict whether eyes are open or closed based on brain activity measurements.

5.1 Model Initialization

The first step in implementing logistic regression is to initialize the model by creating an instance of the LogisticRegression class from sklearn.linear_model. We import the necessary library and instantiate the model with appropriate parameters.

```
from sklearn.linear_model import LogisticRegression

# Initialize the Logistic Regression model

model = LogisticRegression(random_state=42, max_iter=1000)
```

Parameters:

- random_state=42: Ensures reproducibility by fixing the random seed for consistent results across multiple runs
- max_iter=1000: Sets the maximum number of iterations for the optimization algorithm to converge. The default is often too low for complex datasets, so we increase it to ensure the model has sufficient iterations to find the optimal coefficients

The logistic regression model uses the sigmoid function to map input features to probabilities between 0 and 1, with a decision threshold (typically 0.5) to classify observations into binary categories.

5.2 Model Training

After initializing the logistic regression model, we train it on our prepared training data using the fit() method. The training process involves the model learning the optimal coefficients (weights) for each feature that best separate the two classes (eyes open vs. eyes closed) by minimizing the logistic loss function.

```
# Train the model on the scaled training data

model.fit(X_train_scaled, y_train)
```

During training, the logistic regression algorithm iteratively adjusts the feature coefficients to maximize the likelihood that the training data is correctly classified. The algorithm uses optimization techniques (such as gradient descent or L-BFGS) to find the parameter values that minimize classification errors. The fit() method takes two arguments:

- X_train_scaled: The scaled training features (EEG sensor readings)
- y_train: The corresponding training labels (eye states: 0 or 1)

The model learns a linear combination of input features and applies the sigmoid function to output probabilities. Features with stronger correlations to eye state (such as occipital channels O1 and O2 related to visual processing) will receive higher coefficient weights, while less relevant features will have smaller weights.

Once training is complete, the model has learned the decision boundary that best separates the two eye state classes in the feature space. We can verify successful training by checking the model's convergence status and examining the learned coefficients, which indicate the importance and direction of each feature's contribution to the prediction.

5.3 Making Predictions

After successfully training the logistic regression model on the training data, we use the trained model to make predictions on the test set to evaluate its performance on unseen data. The predict() method applies the learned decision boundary to classify new observations based on their feature values.

5.3.1 Prediction on Test Set

We generate predictions for the test set using the predict() method, which takes the scaled test features as input and outputs binary class predictions (0 or 1) for each observation.

```
# Make predictions on the scaled test data

y_pred = model.predict(X_test_scaled)
```

The predict() method works by:

1. Computing the linear combination of features using learned coefficients
2. Applying the sigmoid function to obtain probabilities
3. Classifying observations as 1 (eyes open) if probability ≥ 0.5, otherwise 0 (eyes closed)

The resulting y_pred array contains the predicted eye states for all observations in the test set, which we will compare against the actual values (y_test) to evaluate model accuracy.

5.3.2 Comparing Actual vs Predicted Values

To visualize and assess the model's predictions, we create a comparison DataFrame that displays the actual eye states alongside the predicted values. This allows us to inspect individual predictions and identify any patterns in misclassifications.

```
# Create a DataFrame comparing actual and predicted values

comparison = pd.DataFrame({'Actual value': y_test, 'Predicted value': y_pred})

comparison.head()
```

This comparison table provides a clear view of how well our logistic regression model is performing at the individual observation level. By examining the actual versus predicted values, we can see where the model makes correct predictions and where it fails, providing insights into model performance before conducting formal evaluation metrics.

6. Model Evaluation

Model evaluation is a critical stage in the machine learning pipeline, as it measures how well the trained logistic regression model performs on unseen data. In this section, we evaluate the effectiveness of our eye state classification model using multiple standard evaluation metrics, ensuring a comprehensive assessment of its predictive performance. The evaluation is performed on the test dataset to reflect real-world generalization capability

6.1 Confusion Matrix

To evaluate the classification performance of the logistic regression model, we use a **confusion matrix**, which provides a detailed breakdown of correct and incorrect predictions made by the model

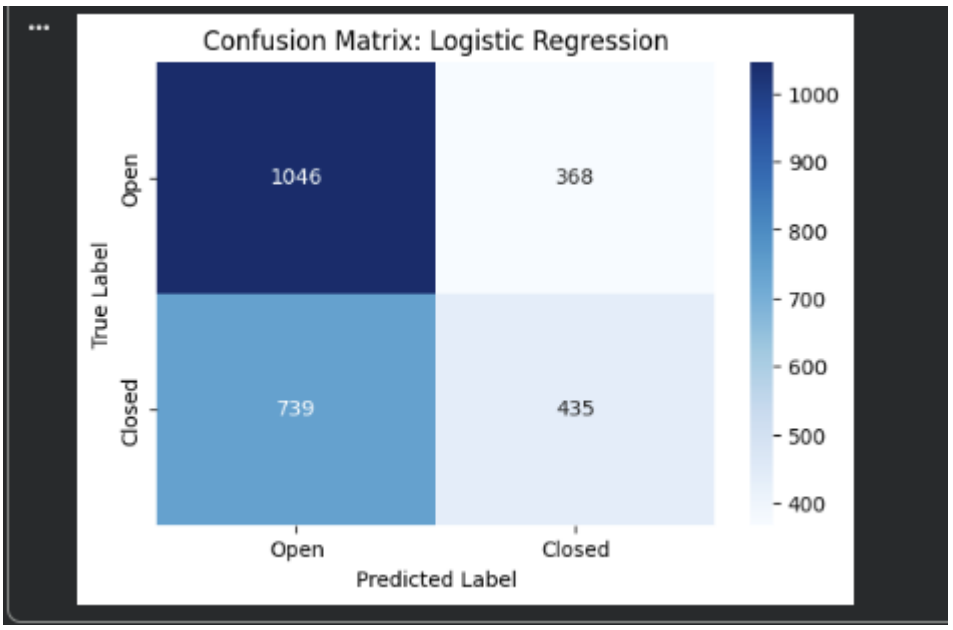
on the test dataset. The confusion matrix compares the **true labels** (y_test) with the **predicted labels** (y_pred) generated using the trained model and the scaled test features (X_test_scaled).

We first generate predictions on the test set using the predict() method of the trained logistic regression model. These predictions are then passed to the confusion_matrix() function from sklearn.metrics to compute the matrix. The resulting confusion matrix contains four key components:

- **True Positives (TP):** Correctly predicted "Closed" eye states
- **True Negatives (TN):** Correctly predicted "Open" eye states
- **False Positives (FP):** "Open" eye states incorrectly predicted as "Closed"
- **False Negatives (FN):** "Closed" eye states incorrectly predicted as "Open"

To enhance interpretability, the confusion matrix is visualized using a heatmap created with seaborn.heatmap(). The heatmap displays the frequency of predictions with clearly labeled axes, where the x-axis represents the predicted labels and the y-axis represents the true labels. The color intensity highlights the distribution of correct and incorrect classifications, making it easier to identify patterns and misclassification trends.

This visualization provides valuable insight into how well the logistic regression model distinguishes between open and closed eye states and helps identify areas where the model may be making systematic errors. The confusion matrix serves as a foundational evaluation tool that supports further analysis using metrics such as precision, recall, and F1-score.



6.2 Classification Report

The classification report is generated using the classification_report() function from sklearn.metrics, which compares the true labels (y_test) with the predicted labels (y_pred). The report evaluates the

model separately for both eye state classes—Open and Closed—and includes the following key metrics:

- Precision: Measures how many of the instances predicted as a particular class are actually correct. High precision indicates a low false positive rate.
- Recall: Measures how well the model identifies all actual instances of a class. High recall indicates a low false negative rate.
- F1-score: The harmonic mean of precision and recall, providing a balanced measure of model performance, especially useful when evaluating classification reliability.
- Support: Indicates the number of true samples available for each class in the test set.

In addition to class-wise metrics, the report also provides:

- Overall accuracy, representing the proportion of correctly classified instances across the entire test set
- Macro average, which treats all classes equally
- Weighted average, which accounts for class support and is particularly relevant when datasets are imbalanced

The classification report offers a comprehensive quantitative evaluation of the logistic regression model, allowing us to assess not only overall accuracy but also how effectively the model performs for each individual eye state. Together with the confusion matrix, this analysis provides a clear understanding of the model's strengths and limitations in EEG-based eye state detection.

```
...                precision    recall  f1-score   support

           Open       0.59      0.74      0.65      1414
         Closed       0.54      0.37      0.44      1174

       accuracy                           0.57      2588
      macro avg       0.56      0.56      0.55      2588
   weighted avg       0.57      0.57      0.56      2588
```

6.3 Accuracy Score

The accuracy score is computed using the accuracy_score() function from sklearn.metrics, which compares the predicted labels (y_pred) with the true labels (y_test). The resulting value represents the fraction of correct predictions made by the logistic regression model on the test dataset.

A higher accuracy value indicates better overall classification performance. However, accuracy is interpreted alongside other metrics such as precision, recall, and F1-score to ensure that strong performance is not achieved at the cost of poor class-wise predictions. In this study, accuracy serves as a high-level indicator of how effectively the logistic regression model can distinguish between open and closed eye states using EEG signals.

```
# ## Evaluate Model: Accuracy Score
from sklearn.metrics import accuracy_score

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy: {accuracy:.4f}")

Model Accuracy: 0.5723
```

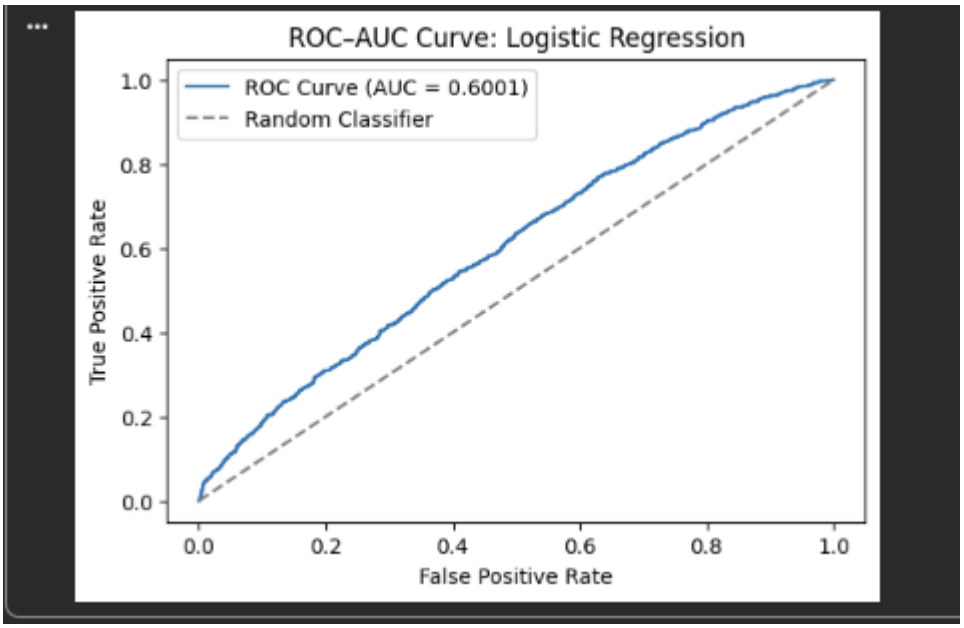
6.4 ROC–AUC Curve

To construct the ROC curve, we first obtain the predicted probabilities for the positive class (eye state = Closed) using the `predict_proba()` method. These probabilities represent the model's confidence in assigning a sample to the positive class. The ROC curve is then generated using the `roc_curve()` function, which calculates the False Positive Rate (FPR) and True Positive Rate (TPR) at various classification thresholds.

The **Area Under the Curve (AUC)** is computed using `roc_auc_score()`. The AUC value ranges from 0 to 1, where:

- A value of **0.5** indicates no discriminative power (random guessing)
- A value closer to **1.0** indicates excellent class separation

The ROC curve visually demonstrates the model's ability to distinguish between open and closed eye states across different thresholds. A curve that lies closer to the top-left corner of the plot indicates better classification performance. The ROC–AUC analysis is particularly useful for evaluating model robustness and comparing classifiers independent of a fixed decision threshold.



7. Performance Analysis and Discussion

After applying comprehensive data preprocessing and training the logistic regression model, the model's performance was evaluated on the test dataset using multiple classification metrics. The evaluation results provide insight into how effectively logistic regression can classify eye states using EEG signals after preprocessing and feature engineering.

The logistic regression model achieved an **overall accuracy of 57%** on the test dataset. This accuracy indicates that more than half of the EEG samples were correctly classified as either eyes open or eyes closed. While the accuracy is moderate, it reflects the inherent complexity of EEG signal data and highlights the challenge of distinguishing subtle neurological patterns using a linear classification model such as logistic regression.

Analysis of the confusion matrix shows that the model performs better in identifying the **Open eye state** compared to the **Closed eye state**. This observation is supported by the classification report, where the **Open** class achieved a **precision of 0.59**, **recall of 0.74**, and **F1-score of 0.65**. The relatively high recall value indicates that the model is effective at correctly identifying most instances where the eyes are open. However, the precision value suggests that some predictions of the open state include incorrect classifications.

For the **Closed** eye state, the model achieved a **precision of 0.54**, **recall of 0.37**, and **F1-score of 0.44**. The lower recall value indicates that the model struggles to correctly identify a significant portion of closed-eye instances, leading to a higher number of false negatives. This imbalance in class-wise performance suggests that EEG patterns corresponding to closed-eye states may be more difficult to separate linearly, or may overlap more with open-eye patterns in the feature space.

The **macro average F1-score of 0.55** indicates balanced but moderate performance across both classes, treating each class equally regardless of sample size. The **weighted average F1-score of 0.56** further confirms consistent performance while accounting for class support, with 1,414 samples for the Open class and 1,174 samples for the Closed class. These results show that the model does not heavily bias one class based on frequency, but its predictive strength varies by eye state.

Overall, the evaluation results demonstrate that logistic regression, when combined with extensive preprocessing and feature engineering, is capable of learning meaningful patterns from EEG data but has limitations in capturing the complex, non-linear characteristics of brain signals. The achieved accuracy of 57% and class-wise performance metrics indicate that logistic regression serves as a reasonable baseline model for EEG-based eye state detection. Future work could improve performance by incorporating more advanced models such as support vector machines, ensemble methods, or deep learning architectures that are better suited for modeling complex EEG signal dynamics.